MF

CASE STUDY

PUBLISHING / HUMANITIES

Scholia — AI Editing & Translation Workspace

An AI editing and translation workspace for an editorial team in the humanities, supporting multilingual content through multi-agent translation, grammar and style review, reference checking, and human-in-the-loop editing.

KEY METRIC

15 hours*

saved per peak week

9%*

Bounce-back rate, down from 33%

1

Instead of 4 separate tools

* Estimated impact based on workflow analysis and pilot observations.

Manos Fragkiadakis

AI Engineer

PUBLISHING / HUMANITIES

15 hours saved per peak week.

PROJECT
Scholia

CLIENT
Humanities research institute

INDUSTRY
Publishing / Humanities

ROLE
AI Engineer

CONTEXT
A small editorial team preparing ~16 articles a week across languages, with translation, grammar and reference work scattered over separate tools — and 33% of articles bouncing back for rework.

Challenge

The team prepared on average 16 articles a week ahead of conferences and volume releases. Editing happened in Word, but translation, grammar, and reference work lived in separate tools and subscriptions. Generic translation tools did not respect the discipline's terminology, the journal's style, or its formatting conventions, so small issues accumulated and reached review: 33% of articles came back needing rework, and during peak weeks the work spilled into evenings and Saturdays.

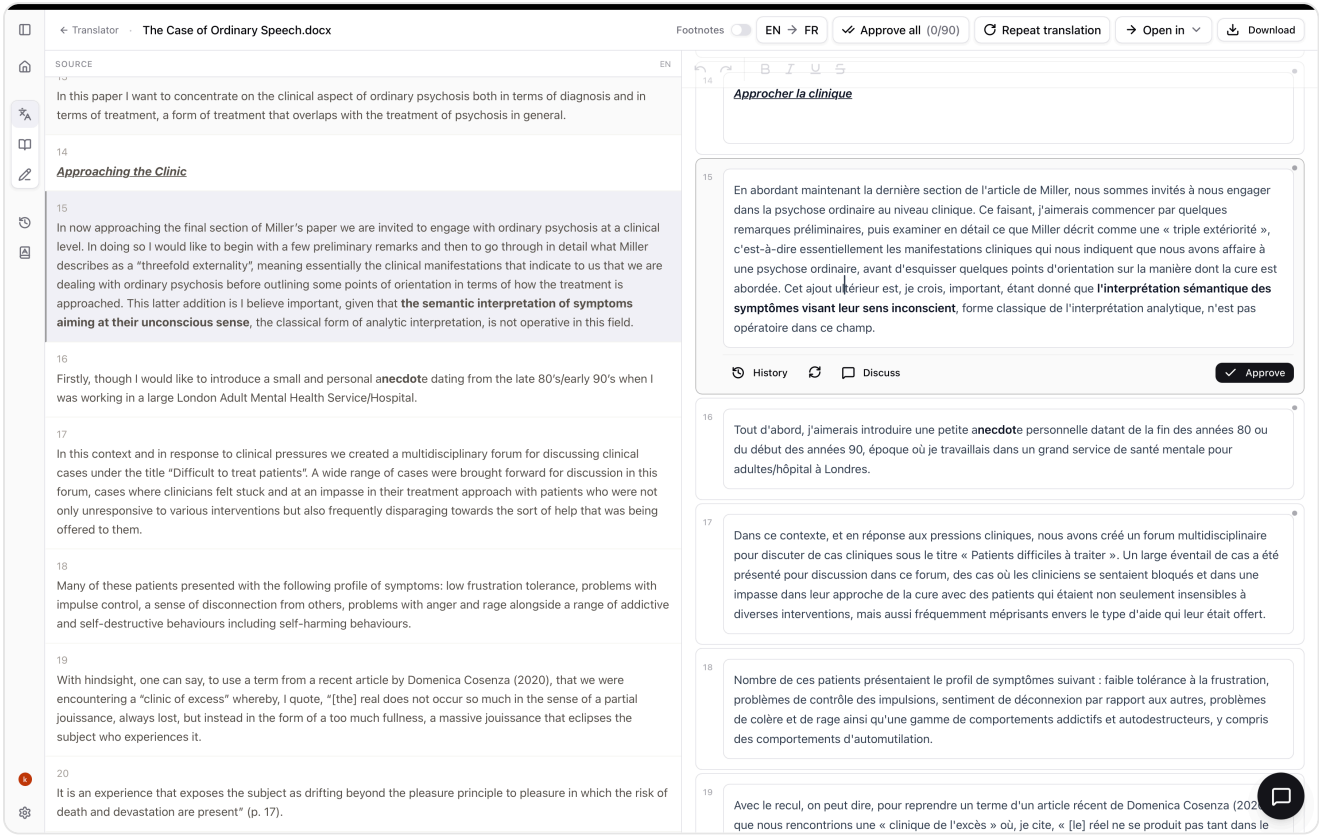
Approach

I built Scholia — AI editing and translation software shaped around the way the team prepares each article.

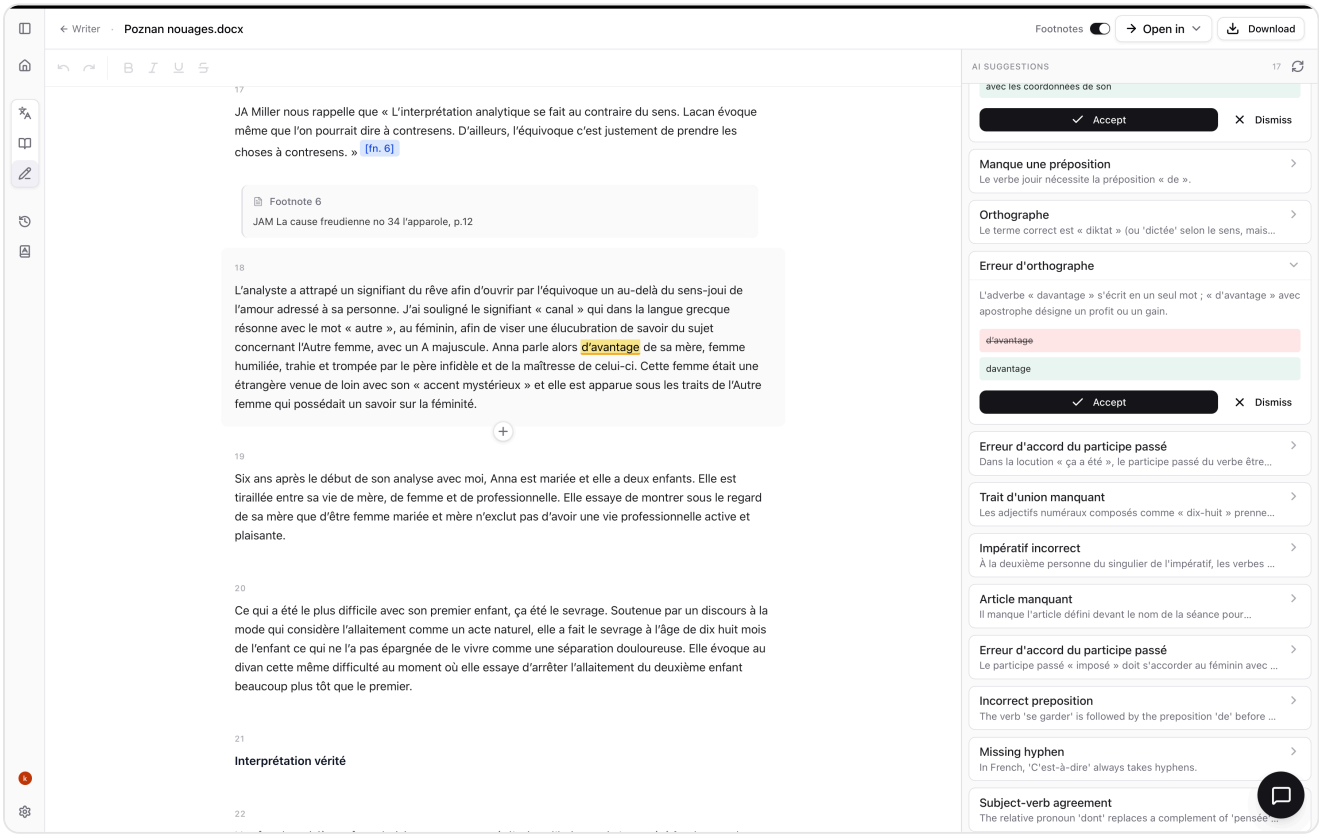
Translation runs as a translator → reviewer → human editor chain. A translator agent translates the article segment by segment, using a custom glossary tuned to the discipline's terminology and the journal's style, and leaves a short note wherever it made a judgment call. A reviewer agent then reads the whole translation for consistency of terms and style and corrects what the translator got wrong. The human editor approves each segment into the working draft, edits it, or leaves feedback. Feedback goes back to the reviewer with the segment's full history, and anything it changes returns to the human editor for approval.

During the main edit it flags grammar, expression, and style issues inline, cross-checks references and claims against the team's uploaded sources, and answers questions about the document through a cited Q&A assistant. **The editor stays in the loop on every change.**

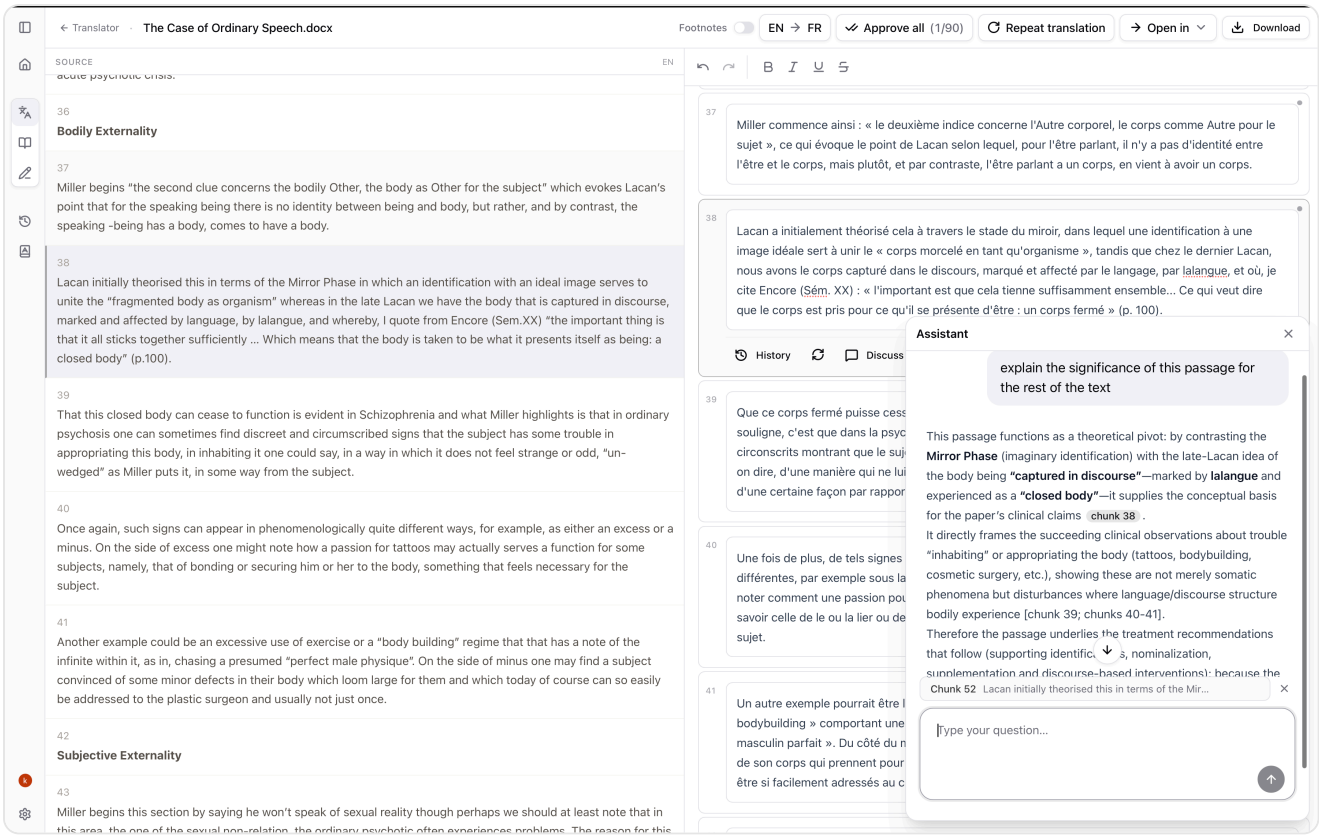
Solution Overview



Document-level translation. The editor approves each segment into the working draft.



Inline grammar suggestions in the main edit. The editor accepts, edits, or rejects each one.



Ask the Assistant about a passage or idea. It draws on the article and the uploaded literature.

How It Works

NLP | Agentic editing | Multi-agent translation | Human-in-the-loop | Custom glossary

Tech Stack

FastAPI | LangGraph | Next.js | PostgreSQL | Azure

Results & Impact

The software saved 15 hours per peak week, without adding headcount. Bounce-back dropped from 33% to 9%, and peak weeks no longer required evenings and Saturdays. The editors got their focus back, with always-on AI handling the repetitive, error-prone work.

15 hours*
Saved per peak week

9%*
Bounce-back rate, down from 33%

1
Instead of 4 separate tools

* Estimated impact based on workflow analysis and pilot observations.

> *I came in skeptical about AI. But Manos built this so every change goes through an editor and the AI doesn't make changes without approval.*
>
> MH **Editorial Director**
> Humanities research institute

Project Details

LOCATION Brussels, Belgium	TIMELINE 2 months (Q1 2026)	AGENT PLATFORM Microsoft Foundry
TEAM SIZE Solo	ENGAGEMENT MODE Direct with the editorial team	

ABOUT MANOS

Manos Fragkiadakis is an Applied AI Engineer building document intelligence and AI knowledge systems. He has over five years of Python software development experience and has built custom ERP and CRM systems for professionals. His recent work includes cited RAG across 3,000+ page legal case files and court-monitoring automation software used by 10+ law offices. He previously worked as an AI engineer and taught Natural Language Processing at Utrecht University.

GitHub _d dunderscore Technical blog